

GEOMETRÍA COMPUTACIONAL

Sesión 30

Reporte técnico del proyecto final

→ NOTAS DE CLASE

Índice

1	1854, 1967, 1979: los mapas como problemas de ingeniería	2
2	Recap: lo que construiste	2
3	El reporte como examen alternativo	3
4	Estructura del reporte	3
4.1	Portada	3
4.2	Resumen	3
4.3	Introducción	4
4.4	Marco teórico	4
4.5	Diseño del sistema	4
4.6	Implementación	4
4.7	Resultados y demostración	5
4.8	Conclusiones	5
4.9	Bibliografía	5
5	Los resultados de aprendizaje en tu proyecto	5
5.1	RA-C1: Soluciones creativas para problemas complejos	6
5.2	RA-C2: Consideraciones más allá del algoritmo	6
5.3	RA-C3: Herramientas modernas y sus limitaciones	7
6	Marco teórico: qué investigar	8
6.1	Temas obligatorios	8
6.2	Temas recomendados	8
6.3	Fuentes de partida	8
7	Las capturas de pantalla que necesitas	9
8	Lo que distingue un buen reporte	9
9	Ejercicios de planeación	10

1854, 1967, 1979: los mapas como problemas de ingeniería

En septiembre de 1854, el médico John Snow clavó pines en un mapa de Londres para marcar los casos de cólera que había documentado durante la epidemia que asolaba el barrio de Soho. Cuando terminó, el patrón era imposible de ignorar: todos los casos se agrupaban alrededor de una sola bomba de agua en la calle Broad. Snow convenció a las autoridades de retirar la manija de esa bomba, la epidemia cedió, y de paso inventó el análisis espacial [1].

Un siglo después, en 1967, el geógrafo Roger Tomlinson creó el primer **Sistema de Información Geográfica** (*Geographic Information System*, GIS) del mundo para inventariar el uso del suelo en Canadá. El reto central que tuvo que resolver fue exactamente el que tú acabas de atacar en este curso: ¿cómo combinar múltiples **capas** de información geográfica en una representación coherente? Tomlinson lo hizo con papel, escáneres primitivos y años de trabajo [2].

Doce años después, en 1979, Jon Bentley y Thomas Ottmann publicaron el artículo que lleva su nombre: un algoritmo capaz de detectar todas las intersecciones entre n segmentos en tiempo $O((n + k) \log n)$, donde k es el número de intersecciones [3]. Ese algoritmo es el núcleo computacional de cualquier sistema que fusione capas geográficas de manera eficiente. Lo implementaste.

Tu proyecto no fue un ejercicio académico abstracto: fue la reimplementación de una pieza fundamental de la infraestructura de software que hace posibles los mapas digitales modernos, desde OpenStreetMap hasta los sistemas de navegación del teléfono.

Recap: lo que construiste

Antes de hablar del reporte, conviene tener claro exactamente qué comprende el sistema. Cada componente tiene nombre, algoritmo y complejidad; todos deben aparecer en tu marco teórico.

1. **Lectura de *layers* en formato DCEL.** El sistema lee archivos que describen una **lista doblemente conectada de aristas** (*Doubly Connected Edge List*, DCEL): vértices, semi-aristas y caras. Es el formato estándar para representar subdivisiones planares.
2. **Fusión de capas con Bentley-Ottmann.** El algoritmo de barrido de plano (*plane sweep*) detecta todas las intersecciones entre segmentos de distintas capas. Cada intersección genera nuevos vértices y divide las aristas involucradas.
3. **Reconstrucción de aristas y caras.** A partir de los vértices e intersecciones calculados se reconstruyen las aristas de la subdivisión resultante y se identifican sus caras, incluyendo la **cara externa** (la cara no acotada).
4. **Exportación de archivos.** El sistema genera archivos de vértices, aristas y caras que describen la nueva capa fusionada en formato DCEL, reutilizable por cualquier otro equipo.

5. **Visualización interactiva.** Una aplicación carga el resultado y permite visualizar el mapa completo, seleccionar y deseleccionar caras individuales (incluida la cara externa) con retroalimentación visual.
6. **Interoperabilidad entre equipos.** Cada equipo subió ejemplos de *layers*; tu sistema debía procesar correctamente los ejemplos de los demás equipos.

El reporte como examen alternativo

Si tu proyecto procesa y visualiza correctamente los *layers* de todos los demás equipos, tienes la opción de entregar un **reporte técnico** en lugar del examen final.

✓ Condición de elegibilidad

El reporte solo aplica si el proyecto funciona correctamente, incluyendo la demostración de que puede procesar los ejemplos entregados por los otros equipos. Un proyecto parcialmente funcional no califica para esta opción.

El reporte es un documento académico-técnico, similar a los que los ingenieros producen en la industria para documentar sistemas que han construido. No es una bitácora de lo que hiciste, ni un resumen de la materia: es la documentación formal de una solución a un problema de ingeniería.

Un examen mide si puedes recordar y aplicar conceptos bajo presión. Un buen reporte técnico demuestra algo más valioso: que entiendes lo que construiste, que puedes conectarlo con la literatura existente y que eres capaz de comunicarlo con claridad y precisión. En la industria, esa es la habilidad que diferencia a los ingenieros excepcionales de los competentes.

Estructura del reporte

El reporte debe contener las siguientes secciones en este orden. Para cada una se indica qué incluir y a qué profundidad.

• Portada

Nombre del proyecto, integrantes del equipo, curso, institución y fecha. Sin colores extravagantes ni adornos innecesarios: una portada limpia comunica profesionalismo.

• Resumen

Un párrafo de **máximo 250 palabras** que responda:

- ¿Qué problema resuelve el sistema?
- ¿Qué algoritmos y estructuras de datos se utilizaron?
- ¿Qué resultados se obtuvieron?

El resumen es lo primero que lee cualquier evaluador. Debe ser auto-contenido: quien lo lea sin conocer el proyecto debe entender de qué trata.

• Introducción

Aquí se establece el contexto y la motivación del trabajo. Esta sección **debe** abordar explícitamente los resultados de aprendizaje RA-C1 y RA-C3 (ver sección 5).

Una introducción completa contiene:

1. **Contexto:** ¿en qué dominio existe el problema? (sistemas GIS, cartografía digital, superposición de capas geográficas).
2. **Problema:** ¿cuál es el reto de ingeniería que aborda el sistema? Sé específico: no es “hacer un mapa”, sino “fusionar múltiples subdivisiones planares detectando todas las intersecciones de forma eficiente y reconstruyendo la topología de la capa resultante”.
3. **Solución propuesta:** qué algoritmos y estructuras de datos se seleccionaron y por qué. Aquí va la conexión con RA-C3.
4. **Alcance:** qué hace y qué no hace el sistema.
5. **Organización del documento:** un párrafo breve que describe qué contiene cada sección.

• Marco teórico

Investigación bibliográfica de los fundamentos del sistema. Ver sección 6 para los temas concretos que debes cubrir y las fuentes de partida.

• Diseño del sistema

Describe la arquitectura del sistema: módulos, flujo de datos y las decisiones de diseño más importantes. Un diagrama de componentes o de flujo de datos es casi obligatorio aquí.

Para cada decisión de diseño relevante responde: ¿por qué lo hiciste así y no de otra manera? Por ejemplo: ¿qué representación elegiste para la cola de eventos de Bentley-Ottmann? ¿Cómo estructuraste la DCEL en memoria? ¿Cómo identificas la cara externa?

• Implementación

Fragmentos de código relevantes con su explicación. **No copies todo el código fuente**; selecciona las partes más interesantes o más difíciles: la estructura DCEL, el manejo del árbol de barrido, la identificación de caras, la lógica de selección en la interfaz.

Un reporte que solo muestra código sin explicarlo no demuestra comprensión. Para cada fragmento incluido, explica qué hace, por qué lo diseñaste así y qué casos límite maneja o no maneja.

• Resultados y demostración

Capturas de pantalla que evidencian el funcionamiento del sistema. Ver sección 7 para la lista específica de capturas requeridas.

Esta sección también es el lugar natural para abordar RA-C2: ¿en qué aplicaciones reales podría usarse el sistema? ¿Qué implicaciones tiene su uso? (ver sección 5).

• Conclusiones

Las conclusiones **deben** retomar explícitamente los tres resultados de aprendizaje. Una buena conclusión no repite el resumen: reflexiona sobre lo que aprendiste, qué salió como esperabas y qué no, y cuál es la relevancia más amplia del trabajo.

Estructura sugerida:

1. Resumen de lo logrado (dos o tres oraciones).
2. RA-C1: reflexión sobre la complejidad del problema y las decisiones creativas tomadas.
3. RA-C2: implicaciones del sistema en aplicaciones reales, medio ambiente o sociedad.
4. RA-C3: limitaciones concretas de los algoritmos y herramientas utilizados.
5. Trabajo futuro: ¿qué mejorarías en una siguiente versión?

• Bibliografía

Mínimo cinco fuentes académicas en formato IEEE o APA, consistente a lo largo del documento. Al menos una fuente debe ser el artículo original de Bentley y Ottmann (1979).

✓ Citar es parte del trabajo técnico

Cada afirmación técnica en el marco teórico debe tener una cita. Escribir “el algoritmo de Bentley-Ottmann tiene complejidad $O((n + k) \log n)$ ” sin citar la fuente es como presentar resultados sin datos. No cuentes páginas de Wikipedia ni repositorios de código como fuentes académicas.

Los resultados de aprendizaje en tu proyecto

Tu reporte debe conectar explícitamente el trabajo realizado con los resultados de aprendizaje de la materia. Esto no es decorativo: es la evidencia concreta de que tu formación como ingeniero avanzó con este proyecto.

Los tres resultados de aprendizaje (**RA**) que debes abordar son:

RA	Enunciado
C1	Diseñar soluciones creativas para problemas complejos de ingeniería.
C2	Diseñar sistemas, componentes o procesos considerando adecuadamente la salud y seguridad públicas, el costo total de vida, el carbono neto cero, y aspectos de recursos, culturales, sociales y ambientales.
C3	Crear, seleccionar, aplicar y reconocer las limitaciones de técnicas y herramientas modernas de TI e ingeniería, incluyendo matemáticas conceptuales, análisis numérico, análisis de datos y aspectos formales de informática y ciencias de la información.

• RA-C1: Soluciones creativas para problemas complejos

Un “problema complejo de ingeniería” no es solo uno difícil: es uno que requiere integrar conocimiento de múltiples áreas, cuyas soluciones no son únicas y cuyas restricciones pueden estar en conflicto [5]. Tu proyecto encaja exactamente ahí:

- **Integración de áreas.** El sistema combina geometría computacional (intersecciones de segmentos), estructuras de datos (DCEL, árboles de búsqueda balanceados), algoritmos de barrido, diseño de formatos de archivo e interfaces gráficas. Ninguna de estas piezas sola resuelve el problema.
- **Soluciones no únicas.** Hay múltiples maneras válidas de representar la DCEL en memoria, de manejar la cola de eventos en el barrido y de identificar las caras resultantes. Las decisiones que tomaste definen tu solución.
- **Restricciones en conflicto.** La interoperabilidad (que otros equipos puedan usar tus archivos) añade una restricción de diseño real: no basta con que funcione para tus propios datos.

En la introducción explica por qué este es un problema complejo: los ingredientes técnicos que confluyen y qué lo hace no trivial de resolver. Evita frases genéricas como “es complejo porque requiere mucho código”; sé específico en los algoritmos y la topología involucrados.

En las conclusiones reflexiona sobre las decisiones de diseño: ¿cuál fue la más difícil? ¿Qué te sorprendió? ¿Cambiarías algo si empezaras de nuevo?

• RA-C2: Consideraciones más allá del algoritmo

Este resultado de aprendizaje pide que pienses en las implicaciones de lo que construiste más allá de su corrección técnica. Para un sistema de fusión de capas geográficas esto es especialmente rico:

Salud y seguridad pública. Los sistemas GIS son infraestructura crítica en emergencias: mapas de zonas de inundación, rutas de evacuación, ubicación de hospitales y estaciones de bomberos. Un error en la fusión de capas en ese contexto puede tener consecuencias físicas reales para las personas.

Carbono neto cero. El análisis espacial es una herramienta clave en la planificación de energía renovable (ubicación de parques solares y eólicos), el monitoreo de deforestación y la modelación del ciclo del carbono a nivel territorial. Tu sistema procesa exactamente el tipo de datos que esas aplicaciones requieren.

Recursos y costo total de vida. Los formatos de datos abiertos e interoperables, como el que diseñaron entre equipos, reducen el costo de integración de sistemas a largo plazo. Un formato propietario atrapado en una empresa tiene costo oculto que se acumula durante años.

Consideraciones culturales y sociales. Los mapas no son neutrales: definen fronteras, derechos sobre la tierra y territorios. Una herramienta de fusión de capas que introduce errores geométricos puede invalidar argumentos legales o técnicos con consecuencias reales para comunidades.

En la sección de resultados o en las conclusiones, desarrolla **dos o tres** de estos puntos (los que mejor conecten con tu implementación concreta) en un párrafo cada uno. La profundidad importa más que el número de puntos cubiertos.

• RA-C3: Herramientas modernas y sus limitaciones

Este es posiblemente el resultado de aprendizaje más concreto de documentar, porque las limitaciones de tu implementación son evidencia directa de él. No las evites: reconocer las limitaciones de las herramientas que usas es una señal de madurez técnica, no de debilidad.

Herramientas y técnicas que aplicaste y que debes documentar:

- **La DCEL como estructura de datos formal.** ¿Cuándo es eficiente y cuándo no? ¿Qué operaciones soporta en $O(1)$ y cuáles requieren recorridos?
- **Algoritmo de Bentley-Ottmann.** Complejidad teórica $O((n+k) \log n)$, más eficiente que el algoritmo ingenuo $O(n^2)$ cuando $k \ll n^2$. ¿Pero cuándo el algoritmo ingenuo sería preferible en la práctica?
- **Precisión numérica.** El algoritmo de barrido asume aritmética exacta, pero los números de punto flotante tienen error de redondeo. Las configuraciones degeneradas (tres segmentos que se intersectan en un punto, segmentos colineales) requieren manejo especial. ¿Cómo lo abordaste o qué casos decidiste no manejar?
- **El kit de interfaz gráfica elegido.** ¿Qué puede y qué no puede hacer? ¿Qué limitó la implementación de la visualización?

En la introducción menciona qué herramientas y técnicas formales utilizaste y por qué las seleccionaste sobre alternativas.

En las conclusiones lista de manera honesta las limitaciones que encontraste: casos que tu implementación no maneja, aproximaciones que tomaste, situaciones donde el algoritmo falla. Esto no resta puntos: los suma.

Bentley, J. L., & Ottmann, T. A. (1979). "Algorithms for reporting and counting geometric intersections." *IEEE Transactions on Computers*, 28(9), 643–647.

Vale la pena leerlo. Tiene menos de cinco páginas y es un ejemplo magistral de cómo presentar un algoritmo: motivación del problema, descripción del enfoque, demostración de complejidad, comparación con la solución ingenua. Tu reporte puede seguir esa estructura.

Marco teórico: qué investigar

El marco teórico no es un glosario. No basta poner una definición de DCEL y pasar al siguiente tema. La pregunta guía para cada concepto es: *¿qué necesita entender el lector para poder juzgar si mi implementación es correcta y eficiente?*

• Temas obligatorios

1. **Subdivisiones planares y la DCEL.** ¿Qué es una subdivisión planar? ¿Por qué la DCEL es la representación estándar? Describe sus tres componentes (vértices, semi-aristas, caras) y sus punteros. Explica qué operaciones soporta eficientemente y cuáles no.
2. **El paradigma del barrido de plano.** ¿Cuál es la idea central detrás de los algoritmos de barrido? ¿Qué es la línea de barrido? ¿Qué invariante mantiene el árbol de estado durante el barrido? ¿Qué tipos de eventos existen?
3. **Algoritmo de Bentley-Ottmann.** Descripción detallada: cola de eventos, árbol de estado, los tres tipos de eventos (inicio de segmento, fin de segmento, intersección). Complejidad en tiempo y espacio. Comparación con el algoritmo de Shamos-Hoej.
4. **Identificación de caras en una subdivisión planar.** ¿Cómo se identifican las caras a partir de las semi-aristas? ¿Cómo se distingue la cara externa?

• Temas recomendados

- **Aplicaciones de GIS** que usan fusión de capas: OpenStreetMap, QGIS, ArcGIS. Contextualizar tu sistema en ese ecosistema muestra que entiendes el dominio del problema.
- **Configuraciones degeneradas en Bentley-Ottmann.** El artículo original asume posición general. ¿Qué pasa con vértices compartidos, segmentos colineales o tres segmentos que se intersectan en un solo punto?
- **Librerías existentes.** CGAL (*Computational Geometry Algorithms Library*) tiene implementaciones robustas de DCEL y Bentley-Ottmann. ¿Por qué las implementaste desde cero en lugar de usar CGAL?

• Fuentes de partida

Libro de referencia. de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications* (3.^a ed.). Springer. Los capítulos 2 (DCEL) y 5 (barrido de plano) cubren exactamente los temas de este proyecto. Es el libro de la materia.

Artículo original. Bentley, J. L., & Ottmann, T. A. (1979). Algorithms for reporting and counting geometric intersections. *IEEE Transactions on Computers*, 28(9), 643–647.

Precursor histórico. Shamos, M. I., & Hoey, D. (1976). Geometric intersection problems. *17th Annual Symposium on Foundations of Computer Science*, 208–215. Detecta si hay alguna intersección (sin reportarlas todas) en $O(n \log n)$. Leerlo antes del artículo de Bentley-Ottmann da perspectiva sobre el avance.

Contexto de aplicación. Documentación de CGAL (<https://www.cgal.org>) y la wiki de OpenStreetMap (<https://wiki.openstreetmap.org>) para el contexto de GIS.

Las capturas de pantalla que necesitas

Las imágenes en un reporte técnico no son decoración: son evidencia. Cada captura debe tener un pie de figura descriptivo que explique qué muestra y qué significa. Una imagen sin explicación no demuestra nada.

Lista mínima de capturas requeridas:

1. **Capas individuales antes de la fusión.** Al menos dos capas distintas cargadas por separado, con sus vértices y aristas visibles.
2. **Estado del barrido o capas superpuestas.** Si el sistema puede visualizar el proceso del barrido o las capas superpuestas antes de la fusión, inclúyelo. Hace visible la complejidad del problema.
3. **Resultado de la fusión.** La capa resultante con vértices, aristas y caras correctamente identificados.
4. **Cara externa destacada.** Una captura que muestre claramente que el sistema identifica y puede seleccionar la cara no acotada.
5. **Selección de caras internas.** Demostración de la funcionalidad de selección y desección con retroalimentación visual.
6. **Ejemplo de otro equipo procesado.** Al menos una captura que muestre tu sistema procesando un *layer* diseñado por otro equipo. Identifica de qué equipo es el ejemplo.

Toma las capturas con resolución suficiente para que texto y líneas sean legibles en el PDF. Una captura borrosa no demuestra nada. Si tu interfaz tiene modo oscuro, considera si el modo claro resulta más legible impreso.

Lo que distingue un buen reporte

No todos los reportes que cumplan los requisitos mínimos serán equivalentes. Los criterios que separan un trabajo notable de uno suficiente son:

Claridad conceptual. ¿Se entiende exactamente qué hace el sistema leyendo solo el reporte, sin ver el código? Un lector que conozca geometría computacional pero no haya visto tu implementación debe poder seguir tu diseño sin ambigüedades.

Honestidad técnica. Los reportes que reconocen limitaciones, casos no manejados y compromisos de diseño son más valiosos que los que pretenden que todo funciona perfectamente. En la industria y en la academia se valora saber qué *no* funciona y por qué.

Conexión con la bibliografía. ¿Tu implementación sigue el algoritmo del libro o lo adapta? ¿En qué difiere y por qué? Un buen reporte conecta la implementación concreta con la literatura publicada.

Consistencia en el formato. Notación matemática consistente, figuras numeradas y referenciadas desde el texto, bibliografía completa con el mismo formato a lo largo del documento.

Narrativa coherente. El reporte debe leerse como un documento, no como una colección de secciones independientes. La introducción plantea algo que el cuerpo desarrolla y las conclusiones cierran.

- Pegar el código fuente completo en el reporte.
- Usar capturas de pantalla del código en lugar de fragmentos bien formateados.
- Escribir las conclusiones repitiendo el resumen.
- Incluir fuentes en la bibliografía que no se citan en el texto (ni al revés: citar sin incluir en la bibliografía).
- Usar Wikipedia como fuente académica primaria.

Ejercicios de planeación

Los siguientes ejercicios no requieren código: son para planear las partes más importantes del reporte antes de empezar a escribirlo. Resuélvelos en papel o en un borrador antes de la siguiente sesión.

1. Lee el abstract del artículo original de Bentley y Ottmann (1979) y escribe, con tus propias palabras y en menos de 100 palabras, el problema que resuelve y la idea central de su solución. Esta será la base de tu descripción del algoritmo en el marco teórico.
2. Escribe el primer párrafo de tu introducción. Debe responder: ¿qué problema resuelve tu sistema? ¿Por qué ese problema es complejo desde el punto de vista de la ingeniería? No uses frases genéricas; sé específico sobre los algoritmos y estructuras involucrados. (RA-C1)

3. Enumera **al menos tres limitaciones concretas** de tu implementación. Para cada una indica si es una limitación del algoritmo, de la precisión numérica, de la interfaz u otra fuente. Estas irán en las conclusiones. (RA-C3)
4. Elige **dos** de los cuatro contextos de RA-C2 (salud y seguridad, carbono, recursos, cultural/social) que mejor conecten con tu sistema y escribe un párrafo de cuatro a seis oraciones para cada uno. (RA-C2)
5. Diseña el layout de la sección de resultados: qué capturas incluirás, en qué orden, y qué dirá el pie de figura de cada una. Usa la lista de la sección 7 como punto de partida.

Resumen

- El reporte técnico es el examen alternativo para equipos con proyecto funcional, incluyendo la demostración con los *layers* de los demás equipos.
- La estructura obligatoria es: portada, resumen, introducción, marco teórico, diseño del sistema, implementación, resultados, conclusiones y bibliografía.
- La **introducción** debe abordar RA-C1 (problema complejo y solución creativa) y RA-C3 (herramientas y técnicas seleccionadas).
- Las **conclusiones** deben abordar los tres resultados de aprendizaje: RA-C1 (reflexión sobre la complejidad y las decisiones de diseño), RA-C2 (implicaciones reales del sistema) y RA-C3 (limitaciones encontradas en algoritmos y herramientas).
- RA-C1: argumenta por qué el problema es genuinamente complejo y qué hay de creativo en tu solución.
- RA-C2: conecta el sistema con aplicaciones que involucren salud, medio ambiente, recursos o impacto social; elige dos o tres y desarróllalos con profundidad.
- RA-C3: documenta qué algoritmos y estructuras elegiste, su complejidad teórica y las limitaciones que encontraste en la práctica.
- El marco teórico cubre DCEL, paradigma de barrido de plano, algoritmo de Bentley-Ottmann e identificación de caras.
- Fuente mínima obligatoria: Bentley & Ottmann (1979). Libro recomendado: de Berg et al. (2008), capítulos 2 y 5.
- Cada captura es evidencia: demuestra que el sistema funciona, con pie de figura que explique qué muestra.
- Reconocer limitaciones es parte de RA-C3 y suma, no resta.

Referencias

- [1] Snow, J. (1855). *On the Mode of Communication of Cholera* (2.^a ed.). John Churchill.
- [2] Tomlinson, R. (2012). *Thinking About GIS: Geographic Information System Planning for Managers* (5.^a ed.). Esri Press.
- [3] Bentley, J. L., & Ottmann, T. A. (1979). Algorithms for reporting and counting geometric intersections. *IEEE Transactions on Computers*, 28(9), 643–647.
- [4] de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). *Computational Geometry: Algorithms and Applications* (3.^a ed.). Springer.
- [5] ABET. (2023). *Criteria for Accrediting Engineering Programs, 2023–2024*. ABET.